

Senior Project Final Report

A Cadence-Based Bio-Feedback System for Workout
Optimization

Ronnie Sidhu

Diego Magana

Advisor: Dr. Vladimir Prodanov

Department of Electrical Engineering

California Polytechnic State University, San Luis Obispo



A Senior Project Report Submitted in Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Electrical Engineering

July 2026

Statement of Disclaimer

Since this project is a result of a class assignment, it has been graded and accepted as fulfillment of the course requirements. Acceptance does not imply technical accuracy or reliability. Any use of information in this report is at the risk of the user. These risks may include catastrophic failure of the device or infringement of patent or copyright laws. California Polytechnic State University at San Luis Obispo and its staff cannot be held liable for any use or misuse of the project.

Contents

Statement of Disclaimer	i
Acknowledgments	xi
Abstract	xii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Scope and Constraints	3
1.5 Report Organization	4
2 Background and Related Work	5
2.1 Domain Background	5
2.2 Sensing Technologies Survey	6
2.3 Existing Products and Prior Art	6
2.4 Gap Analysis	6
2.5 Relevant Standards	7
3 System Requirements and Design Constraints	9

3.1	Functional Requirements	9
3.2	Non-Functional Requirements	10
3.3	Design Constraints	11
3.3.1	Economic	11
3.3.2	Environmental	12
3.3.3	Manufacturability	12
3.3.4	Sustainability	13
3.3.5	Ethical	13
3.3.6	Health and Safety	13
3.3.7	Social and Political	14
3.4	Requirements Traceability	14
4	Design	17
4.1	System-Level Design	17
4.1.1	System Architecture Overview	17
4.1.2	Design Alternatives Considered	17
4.1.3	Final Architecture	18
4.2	Hardware Design	20
4.2.1	Sensor Selection and Placement	20
4.2.2	Circuit Design	21
4.2.3	Perfboard Layout	21
4.2.4	Power System	24
4.2.5	Mechanical and Enclosure Design	25
4.2.6	Bill of Materials Summary	26
4.2.7	Design Revisions	28
4.3	Firmware and Embedded Software	29

4.3.1	MCU and Toolchain Overview	29
4.3.2	Sensor Sampling and Data Pipeline	29
4.3.3	Cadence Detection Algorithm	30
4.3.4	Feedback Trigger Logic	33
4.3.5	Communication Protocol	33
4.3.6	Power Management Firmware	34
4.4	Application and User-Facing Software	34
4.4.1	Architecture Overview	34
4.4.2	Feedback Modalities	34
4.4.3	Data Visualization and Logging	35
4.4.4	UX Design Decisions	35
5	Test Plans	37
5.1	Test Methodology	37
5.2	Test Matrix	38
6	Development and Construction	40
6.1	Bench and Unit Testing	40
6.2	Team Roles and Division of Labor	41
6.3	Risks Encountered and Mitigations	42
7	Integration and Test Results	43
7.1	System Integration	43
7.1.1	Hardware-Firmware-App Integration Process	43
7.1.2	Integration Challenges and Resolutions	44
7.2	On-Body Testing Results	44
7.3	Requirements Verification	45

7.4	Results and Discussion	46
7.4.1	Summary of Key Results	46
7.4.2	Interpretation	46
7.4.3	Comparison to Existing Products and Prior Art	47
7.4.4	Unexpected Findings and Failure Modes	48
7.5	Limitations of Testing	48
8	Conclusion and Future Work	50
8.1	Summary of Contributions	50
8.2	Lessons Learned	51
8.3	Future Work	52
	References	53
	Appendices	57
A	Analysis of Senior Project Design	58
B	Full Schematics	64
C	Perfboard Layout Images	65
D	Full Bill of Materials	67
E	Firmware Source Code	69
F	CAD Drawings	78
G	Raw Test Data	79
G.1	PPG Sensor Data	79
G.2	Accelerometer Data	80

H Datasheets Referenced	81
I User Manual / Quick-Start Guide	83

List of Figures

1	System block diagram	18
2	Power Subsystem Schematic	22
3	Perfboard assembly during construction, showing the ESP32, PCM5102 DAC, ADXL335 accelerometer, DC/DC converter, and microSD breakout (unused in the final firmware) on the main board, with the OLED/rotary-encoder board module still unattached at this stage	23
4	Main perfboard with the OLED display and rotary encoder board attached at a right angle to the main board	24
5	Completed 3D-printed enclosure, closed	26
6	Enclosure with sliding lid open, showing the battery seated in place	27
7	Cadence detection and heart-rate-to-cadence control flow	32
8	Measured vs. ground-truth cadence across trials	45
8	Main perfboard assembly	65
9	Main perfboard with UI (OLED/rotary encoder) board attached .	66
11	Main Enclosure Lid	78
12	Raw infrared and red-light measurements collected from the MAX30102 sensor.	79

13	Raw three-axis measurements collected from the ADXL335 accelerometer.	80
----	---	----

List of Tables

I	Comparison of existing cadence-feedback products	7
II	Functional requirements	9
III	Requirements traceability matrix	14
IV	Design alternatives considered per subsystem	19
V	ESP32 pin mapping by subsystem	22
VI	Power budget	25
VII	Bill of materials summary	27
VIII	Planned verification method by requirement	38
IX	Bench test results	41
X	Summary of key results against targets	46
XI	Complete bill of materials	67

Acknowledgments

We would like to thank Dr. Prodanov for providing us with the project, assisting with problem-solving and keeping us on track. We would also like to thank Yvonne Lynch for helping order necessary supplies for the project, as well as the Student Project Lab and Student Makerspace faculty for providing us space to use soldering equipment and 3D printers.

Abstract

Maintaining a target heart-rate zone is central to effective interval and endurance training, yet most consumer wearables only display heart-rate data after the fact, leaving the user to manually adjust pace without any real-time guidance. This project addresses that gap by developing a wearable device that continuously monitors heart rate during a workout and provides real-time auditory feedback to help the user reach and hold a desired heart-rate target.

The system is built around an ESP32 microcontroller paired with a MAX30102 heart-rate sensor and an ADXL335 accelerometer for step cadence. Onboard firmware detects heartbeats from the raw PPG signal, compares the resulting heart rate against a user-specified target, and periodically adjusts the tempo of an audible metronome played through earbuds accordingly. By matching their running cadence to this audio cue, the user can raise or lower their pace — and in turn their heart rate — without needing to glance at a screen or interrupt the workout to check a display.

The completed device was tested under normal workout conditions by the two-person development team, comparing the sensor's heart-rate readings against each team member's own manually counted pulse. Across these trials, the system's readings had a mean absolute error of approximately 4.2 beats per minute, cor-

responding to roughly 93-98 percent accuracy depending on where in the device's 60-200 BPM operating range the reading falls, while auditory cadence updates were generated in real time with no perceptible lag between a change in heart rate and the corresponding audio adjustment. The device runs entirely on battery power, achieving a calculated battery life of approximately 5.8 hours under continuous use.

These results demonstrate that a low-cost, self-contained wearable can deliver usable, real-time heart-rate feedback for cadence-based training without relying on a smartphone app or external display. Because the design uses inexpensive, readily available components and a compact embedded signal-processing pipeline, the approach is practical to reproduce or extend. Future work could include a larger and more diverse user study beyond the development team, refinement of the filtering and cadence-generation algorithm, and additional feedback modalities beyond audio, such as haptic cues.

Chapter 1

Introduction

1.1 Motivation

Cardiovascular exercise is widely recognized as one of the most effective methods for improving overall health, endurance, and cardiac function. Regular aerobic activity reduces the risk of chronic cardiovascular disease, improves metabolic efficiency, and enhances overall physical performance. Structured training approaches, including high-intensity interval training and cadence-regulated running programs, have been shown to maximize physiological adaptations by optimizing exercise intensity, duration, and recovery periods [1]. These benefits are especially important for beginner runners and recreational athletes, who often seek to improve fitness while minimizing the risk of injury; regulated training intensity plays a significant role in both improving performance outcomes and reducing the likelihood of running-related injuries among novice participants [2].

Despite these well-documented benefits, many individuals struggle to exercise effectively due to a limited understanding of how to interpret physiological feedback during a workout. Modern wearable fitness devices commonly report metrics

such as heart rate, cadence, speed, and activity duration, but these measurements typically require the user to interpret numerical data in real time – a task that is cognitively demanding during physical activity. Real-world exercise environments further introduce variability and noise into sensor data, making it harder to derive meaningful insight without additional processing [3]. Biosensor data, while information-rich, can be difficult to use effectively without a feedback system designed to simplify user interaction [4]. As a result, many wearable devices fail to fully support users in maintaining an optimal cardiovascular training zone.

1.2 Problem Statement

Existing wearable fitness devices measure the physiological signals needed to guide effective cardiovascular training, but they largely leave the burden of interpretation to the user. A device that reports a heart-rate number does not, by itself, tell a beginner runner whether to speed up, slow down, or hold pace – and expecting a user to make that judgment mid-workout, from noisy real-time data, works against the goal of accessible, low-friction training.

This project targets that specific gap: translating a continuously measured physiological signal (heart rate) into a simple, actionable cue that a user can follow without consciously interpreting any numbers.

1.3 Project Objectives

- O1. Continuously measure user heart rate during exercise using a wearable PPG sensor, with accuracy within the project’s target of $95\% \pm 5\%$ relative to a manual reference measurement.

- O2. Detect user cadence via an onboard accelerometer and combine it with heart-rate data to determine whether the user’s current effort is above, below, or at their target heart-rate zone.
- O3. Generate a real-time auditory cadence cue, delivered through an earbud, that guides the user toward their target heart rate without requiring them to view a screen.
- O4. Keep the feedback loop responsive, with updates delivered within approximately 5 seconds of a change in measured heart rate.
- O5. Package the system into a compact, comfortable, battery-powered wearable form factor suitable for a full workout session.

1.4 Scope and Constraints

The system is designed for beginner runners and recreational athletes seeking accessible, real-time guidance during cardiovascular training; it is not intended as a diagnostic or medical-grade device. Design work was scoped and constrained by a target unit cost, a compact wearable form factor, and a target battery life sufficient for a single extended workout session (see Chapter 3 for the finalized quantified requirements). The project builds directly on a predecessor design [5], and much of its scope was shaped by addressing that design’s usability shortcomings (see Section 4.1).

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 reviews the physiological and technical background motivating the project and surveys related work. Chapter 3 presents the quantified customer and engineering requirements used to guide the design. Chapter 4 describes the system-level architecture, the design alternatives considered, and the hardware, firmware, and application-software design of the final system. Chapter 5 presents the test methodology and test matrix used to verify the requirements. Chapter 6 covers unit-level bench testing during construction, team roles, and risks encountered along the way. Chapter 7 details system integration and presents the full test results, discussed relative to the project's original objectives. Chapter 8 concludes with a summary of contributions and directions for future work.

Chapter 2

Background and Related Work

2.1 Domain Background

Recent research has explored the dynamic relationship between exercise workload and heart-rate response. Nonlinear modeling techniques have shown that heart rate can be influenced and regulated through controlled adjustments in exercise intensity, providing a theoretical foundation for feedback-based cardiovascular training systems [6]. Building on this, control-system approaches to cadence-heart-rate regulation have illustrated how pacing inputs can be used to guide a user toward a desired cardiovascular state through mathematically defined control strategies [7]. Together, this work provides the physiological and control-theoretic basis for using a cadence-driven audio signal – rather than a raw numeric display – as the feedback mechanism in this project.

2.2 Sensing Technologies Survey

Recent advancements in wearable sensing technology have enabled more accurate monitoring of both internal physiological workload and external biomechanical activity during exercise. Devices incorporating sensors such as photoplethysmography (PPG) for heart-rate measurement and accelerometers for motion detection can provide continuous real-time feedback regarding exercise intensity [8].

2.3 Existing Products and Prior Art

Prior design efforts in wearable cardiovascular training systems have explored feedback mechanisms to help runners maintain appropriate training intensity. Early implementations demonstrated the feasibility of providing real-time guidance through a wearable device, while also revealing design challenges related to usability, feedback clarity, and system integration [5]. Subsequent work further investigated the use of auditory cadence cues as an intuitive method for regulating exercise intensity: real-time cadence feedback was shown to help users maintain a target heart-rate zone by synchronizing physical movement with an externally provided rhythmic signal, reducing the need for conscious interpretation of numerical data [9].

2.4 Gap Analysis

The proposed system differs from the predecessor design primarily in its choice of sensing hardware: replacing a chest-strap heart-rate monitor and a shoe-mounted force-sensitive resistor with a wrist/finger-worn PPG sensor and a hip-mounted accelerometer, respectively. This removes the discomfort and perspiration issues

Table I: Comparison of existing cadence-feedback products

Product	Sensing Method	Feedback Type	Notes
Cardio Trainer [5]	Chest-strap HR + shoe FSR	Auditory cadence	Predecessor design; comfort/wiring issues
This project	PPG + accelerometer	Auditory cadence (single earbud)	Removes chest strap and shoe wiring
Garmin Running Dynamics [10]	Wrist PPG (HR) + chest/torso-strap accelerometer (cadence, form)	On-screen numeric/color-gauge display	Requires glancing at watch; no real-time audio guidance
Polar wrist-based training [11]	Wrist PPG	On-screen target heart-rate zone display	User must read display to know if in-zone
WHOOP [12]	Continuous wrist/arm PPG	App-based coaching, post-workout "Strain" score	Feedback is largely retrospective, not real-time during exercise

associated with a chest strap and eliminates the need for in-shoe wiring, while consolidating the main circuitry into a single hip-mounted enclosure (see Section 4.1 for the full design comparison). This directly addresses prior findings that simplifying physiological feedback into a perceptible, actionable form can improve user adherence to structured cardiovascular training programs.

2.5 Relevant Standards

This project does not target certification against a specific communications or safety standard. The final design does not incorporate wireless communication, so standards such as FCC Part 15 (unlicensed RF devices) or Bluetooth SIG certification do not apply.

Power is supplied by a rechargeable 9V battery, and the system's only battery safety provision is a physical on/off switch; no dedicated overcharge, over-

discharge, or thermal protection circuitry was added beyond what is already built into the battery and its charger. Users should follow the battery manufacturer's charging and handling recommendations. Adding dedicated battery protection circuitry is noted as a possible improvement in Chapter 8.

Chapter 3

System Requirements and Design Constraints

3.1 Functional Requirements

Table II: Functional requirements

ID	Requirement	Priority
REQ-1	The system shall measure the user's heart rate via a PPG sensor.	High
REQ-2	The system shall measure the user's step cadence via an accelerometer.	High
REQ-3	The system shall accept a user-specified target heart rate via a physical control input.	High
REQ-4	The system shall process heart rate and cadence data on an onboard microcontroller to compute a pacing signal.	High
REQ-5	The system shall generate an audible rhythm through earbuds that guides the user's pacing.	High
REQ-6	The system shall be powered by an onboard rechargeable battery supplying all components.	High

3.2 Non-Functional Requirements

The system meets four of its six non-functional targets outright (latency, battery life, weight, and cost); accuracy meets the target at higher heart rates but falls just short at the low end of the operating range (see below), and comfort was not formally verified at all.

Latency. Against a target of 5 seconds, the system achieved real-time cadence updates of approximately 1 second, measured from a change in heart rate to the corresponding change in audio feedback.

Battery life. The system targets a battery life of 6 hours, sufficient for one extended workout session. Based on a combined current draw of 250–300 mA across the system’s 3.3V rail and the 9V-to-5V DC/DC conversion stage, the system achieves a calculated battery life of approximately 5.8 hours under worst-case continuous load (see Section 4.2.4 for the full derivation).

Weight and form factor. Against a target of 12 oz (16 oz maximum), the completed device weighs approximately 10 oz – 8 oz of components plus a 2 oz 3D-printed enclosure.

Cost. Against a target of \$175, the final bill of materials for a single unit totals \$45.20, calculated from one of each core component (ESP32, MAX30102 PPG sensor, ADXL335 accelerometer, earbuds, 9V battery, enclosure, and supporting passive components and hardware); see Table VII and Appendix D for the full breakdown.

Accuracy. Heart-rate accuracy was measured as a mean absolute error (MAE) of approximately 4.2 BPM against a manual pulse-count baseline (Section 6), rather than directly as a percentage. Because the device operates over a user-selectable target range of 60–200 BPM, a fixed BPM error translates to a different

percentage accuracy depending on where in that range the user's heart rate falls: at the low end (60 BPM), a 4.2 BPM error corresponds to $100\% \times (1 - 4.2/60) \approx 93\%$ accuracy, while at the high end (200 BPM), the same absolute error corresponds to $100\% \times (1 - 4.2/200) \approx 98\%$ accuracy. The system's accuracy therefore falls within an approximate 93–98% range across its normal operating zone, which spans the $95\% \pm 5\%$ target – meeting it at higher heart rates and falling just short of it at the low end of the range (see Section 7.5 for discussion).

Comfort. Against a target of $\geq 90\%$ user approval, no formal comfort survey with a meaningful sample size was conducted; comfort was only assessed informally by the two team members, who both found the final design subjectively more comfortable than the predecessor (Section 7.1). This requirement remains unverified rather than passed or failed.

3.3 Design Constraints

3.3.1 Economic

The engineering effort to design, test, and refine the system represents the primary human capital investment; a commercial version would additionally involve assembly labor, component suppliers, and logistics. Natural capital is drawn on through the raw materials used in electronic components and batteries.

If manufactured commercially at a scale of 1,000,000 units per year, with an estimated manufacturing cost of \$25 per unit and a retail price of \$175, the product would generate an estimated \$175,000,000 in gross revenue against \$25,000,000 in manufacturing cost, for an approximate gross profit of \$150,000,000. This is a mass-production estimate and is distinct from the \$45.20 single-unit prototype

cost calculated in Section 3.2; a rechargeable 9V-equivalent power source was estimated at approximately \$3.50 per unit at commercial sourcing volumes, compared to the retail component cost used in the prototype BOM.

3.3.2 Environmental

The primary environmental impact of the device stems from the manufacturing and end-of-life disposal of its electronic components and battery, rather than from its normal operation, which produces no emissions or pollutants. Lithium-ion battery production requires mining of lithium, cobalt, and rare earth elements, which has been associated with habitat destruction and pollution in extraction regions [13]. Semiconductor fabrication for the microcontroller and sensor components similarly contributes industrial waste and energy consumption upstream of assembly. The device’s primary environmental burden therefore occurs at end of life, when it becomes electronic waste.

3.3.3 Manufacturability

The device relies on commercially available electronic components (microcontroller, sensors, audio hardware) that benefit from established consumer-electronics manufacturing infrastructure, which supports scalable production. However, global semiconductor supply constraints could delay sourcing of the microcontroller or sensor components and increase production costs. At scale, manufacturing would additionally involve enclosure production (e.g., injection molding in place of the prototype’s 3D-printed enclosure) and final assembly.

3.3.4 Sustainability

Small consumer electronics are frequently discarded rather than repaired when they fail. Using a rechargeable battery, as this design does, reduces disposable battery waste relative to a single-use alternative, though it introduces additional lithium battery handling considerations (see Health and Safety, below). Designing for component-level replacement – most notably the battery – could extend the product’s usable life and reduce electronic waste, at the cost of additional design and manufacturing complexity.

3.3.5 Ethical

Some materials used in electronic components and batteries are sourced from regions where labor conditions may be unsafe or poorly regulated; the U.S. Department of Labor has identified lithium-ion battery supply chains specifically as an area of child labor and unsafe working condition risk [13]. A commercial version of this product would need to prioritize suppliers who follow ethical labor practices and internationally recognized safety standards.

3.3.6 Health and Safety

Health and safety considerations exist both upstream, in raw material extraction and processing, and in the device itself. Lithium-based batteries are the primary in-device safety concern, as they can become hazardous if damaged, overheated, or improperly charged. Dedicated battery protection circuitry and a safe charging system were identified as a design goal; the current prototype does not implement dedicated protection circuitry beyond a physical on/off switch (see Section 2, Relevant Standards), which is noted as an area for future improvement rather

than a resolved requirement. During use, the device should not pose a hearing-safety risk; audio output levels should remain within safe listening limits during exercise.

3.3.7 Social and Political

Stakeholders include the project’s engineers, potential manufacturers and investors, and the end users – beginner runners and recreational athletes – as well as academic institutions involved in the technology’s development. Wearable, real-time physiological feedback has been associated with improved exercise adherence and cardiovascular training outcomes [14, 15], suggesting a plausible public-health benefit if the device were adopted more broadly. At the same time, production depends on global electronics supply chains that span regions with varying labor and environmental standards, raising the same ethical sourcing concerns discussed above.

3.4 Requirements Traceability

Table III: Requirements traceability matrix

ID	Design Feature	Verification Method	Result
REQ-1	MAX30102 PPG sensor interfaced to the ESP32 via I2C	Compared measured BPM against manual pulse count	~4.2 BPM MAE (~93–98% accuracy across operating range; pass)

ID	Design Feature	Verification Method	Result
REQ-2	ADXL335 accelerometer analog output sampled by ESP32 ADC	Bench tilt/shake testing and on-body step detection testing	Pass
REQ-3	Rotary encoder and OLED display for target heart-rate selection	Functional test of knob input and menu navigation	Pass
REQ-4	ESP32 firmware combining PPG and accelerometer data into a pacing signal	End-to-end testing of pacing output against target heart rate	Pass
REQ-5	PCM5102 DAC driving earbuds with an audible rhythm	Audio output and audibility testing	Pass
REQ-6	9V rechargeable Li-ion battery with DC/DC conversion powering all components	Power budget calculation from measured current draw	Pass (see NFR-2)
NFR-1 (La- tency)	Non-blocking real-time firmware loop	Measured time from heart-rate change to audio update	~ 1 s (target ≤ 5 s)
NFR-2 (Battery life)	9V, 1200 mAh Li-ion battery with 80% efficient DC/DC converter	Calculated from measured current draw and converter efficiency	~ 5.8 hr calculated (target 6 hr)
NFR-3 (Weight)	Perfboard assembly with 3D-printed PLA enclosure	Physical weighing of components and enclosure	~ 10 oz (target 12 oz, 16 oz max)

ID	Design Feature	Verification Method	Result
NFR-4 (Cost)	Consumer-grade components (ESP32, MAX30102, ADXL335, etc.)	Bill-of-materials calculation for a single unit	\$45.20 (target \$175)
NFR-5 (Accu- racy)	MAX30102 signal filtering algorithm	Compared against manual heart-rate measurement	~4.2 BPM MAE (~93–98% depending on HR; target $95\% \pm 5\%$)
NFR-6 (Com- fort)	Finger-worn PPG + hip-mounted enclosure (vs. predecessor’s chest strap + shoe FSR)	Informal comparison by the two team members (no formal survey)	Unverified – subjectively improved, but no $\geq 90\%$ approval data collected

Chapter 4

Design

4.1 System-Level Design

4.1.1 System Architecture Overview

Figure 1 shows the system’s functional architecture. The 9V battery is stepped down to 5V by a DC/DC converter to power the ESP32, which regulates 3.3V for the PPG sensor, accelerometer, OLED display, and rotary encoder. The PPG and accelerometer provide heart-rate and cadence data to the ESP32, which combines them into a pacing signal and outputs it as an audio rhythm through a DAC to the earbuds. The rotary encoder and OLED display allow the user to select a target heart rate and view system status.

4.1.2 Design Alternatives Considered

The design builds directly on a predecessor senior project, CardioTrainer [5], which validated the core concept of cadence-based heart-rate feedback but had significant usability shortcomings. For each of the predecessor’s major subsystems, an

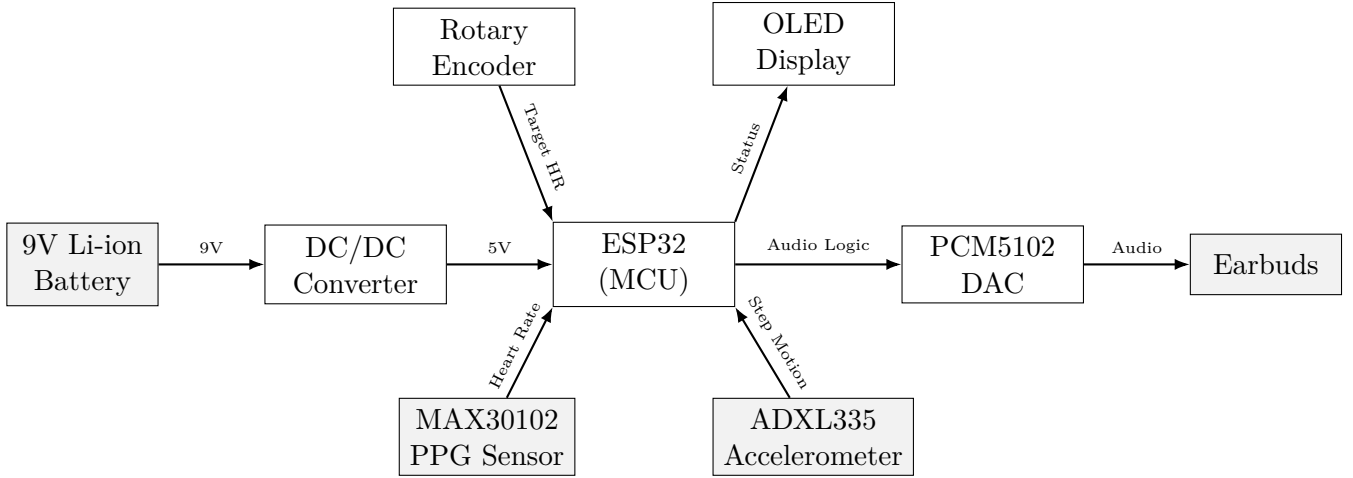


Figure 1: System block diagram

alternative was evaluated against the original approach, as summarized in Table IV.

The PPG sensor was selected over the chest strap primarily for ease of integration and overall simplicity: it requires no additional strap hardware, no chest-level wiring, and interfaces directly with the ESP32 over I2C, whereas the chest strap used by the predecessor project was a separate, self-contained peripheral that the wearer had to don in addition to the rest of the system.

4.1.3 Final Architecture

Moving from a chest-strap-plus-shoe-FSR arrangement to a PPG-sensor-plus-accelerometer arrangement allowed the bulk of the system’s electronics to be consolidated into a single hip-mounted enclosure, with only the PPG sensor itself worn separately on the finger. This final architecture satisfies the comfort, battery-life/power-consumption, and cost requirements identified in Chapter 3: it eliminates the chest strap’s discomfort and \$100 cost, removes the in-shoe FSR wiring, and (via the 9V rechargeable supply) provides sufficient power headroom for the

Table IV: Design alternatives considered per subsystem

Subsystem	Predecessor proach	Ap- proach	Alternative (Chosen)	Rationale
Heart-rate sensing	Chest strap (~\$100)		PPG sensor (~\$11)	Chest straps increase sweat and discomfort during exercise and add significant cost; a PPG sensor integrates more simply into a finger/wrist form factor at a fraction of the cost.
Cadence sensing	Shoe-mounted force-sensitive resistor (FSR)		Accelerometer	The FSR required in-shoe wiring back to the main enclosure. An accelerometer removes that wiring entirely and allows the main circuitry to move from the leg to a single hip-mounted enclosure.
Power supply	3.7V battery		9V rechargeable battery	The 3.7V supply used by the predecessor did not provide enough headroom for this design's power budget; a 9V rechargeable battery was chosen to comfortably supply the DC/DC converter and downstream 3.3V rail.

ESP32 and its peripherals. The System-Level Design in Section 4.1 reflects this final architecture.

4.2 Hardware Design

4.2.1 Sensor Selection and Placement

The design uses two sensors to characterize the user’s exercise state: a MAX30102 photoplethysmography (PPG) sensor for heart rate, and an ADXL335 accelerometer for step cadence.

The MAX30102 is worn on the finger rather than the wrist. Finger placement was chosen over the wrist because it produces the strongest, most artifact-resistant PPG signal: the finger has a high density of capillaries close to the skin surface, which yields a larger, cleaner pulsatile signal than is typically obtainable at the wrist, where the sensor is farther from major vessels and more prone to motion-induced artifacts. This directly supports the system’s accuracy requirement (see Chapter 3), which is the highest-risk specification given how sensitive PPG signals are to motion noise.

The ADXL335 accelerometer is mounted inside the main enclosure along with the rest of the system’s electronics (ESP32, DC/DC converter, DAC), which sits at the hip rather than the leg or shoe. Co-locating the accelerometer with the rest of the electronics – instead of mounting it separately at the shoe, as the predecessor project’s force-sensitive resistor was – avoids running a separate wiring harness down the leg, simplifying construction and improving wearer comfort.

4.2.2 Circuit Design

The circuit is organized around a single power rail cascaded through two regulation stages, feeding four distinct interface groups.

Power subsystem. A 9V battery feeds a 4R7 DC-DC boost step up converter module, which supplies 5V into the ESP32's Vin pin. The ESP32's onboard regulator then produces a 3.3V rail (Vout) that powers every other module in the system. A common ground, shared from the battery through the ESP32's multiple GND pins, ties all subsystems together.

Sensor and display interfaces. The MAX30102 PPG sensor and the OLED display both communicate over I2C, but are placed on two *separate* I2C buses (using the ESP32's dual hardware I2C peripheral) rather than sharing one. A shared-bus configuration was tested first, but produced bus contention where only one device would respond even after assigning each device a distinct hardware address; splitting the two devices onto independent I2C buses resolved this. The ADXL335 accelerometer outputs three analog voltages (X, Y, and Z axes), which are read directly by three of the ESP32's analog input pins. The rotary encoder used to select the target heart rate and start/stop the workout is read via GPIO pins, and the PCM5102 DAC communicates over I2S (bit clock, word-select, and data lines). The complete pin mapping for every subsystem is given in Table V.

4.2.3 Perfboard Layout

The final system is built entirely on perfboard rather than a custom fabricated PCB. Construction is split across two boards: one large perfboard carries the majority of the system – the ESP32, the DC/DC converter, the PCM5102 DAC, and the ADXL335 accelerometer breakout – while a second, smaller perfboard

Table V: ESP32 pin mapping by subsystem

Subsystem	Signal	ESP32 Pin
Rotary Encoder	CLK / DT / SW	GPIO32 / 33 / 25
PCM5102 (I2S)	DAC BCK / WSEL / DATA	GPIO14 / 26 / 27
MAX30102 (I2C bus 0)	PPG SDA / SCL	GPIO21 / 22
ADXL335 Accelerometer	Ac- X / Y / Z	GPIO34 / 35 / 36
OLED Display (I2C bus 1)	SDA / SCL	GPIO18 / 19
Power	VIN / GND	from 9V→5V regulator VOUT+ / VOUT−

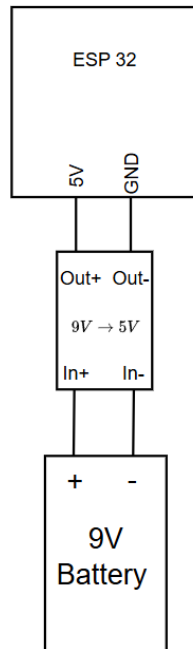


Figure 2: Power Subsystem Schematic

was cut down to size to carry the OLED display and rotary encoder, the two components the user directly interacts with. This second board is mounted to a face of the enclosure so the display and encoder are accessible from outside the case while the rest of the electronics remain enclosed. A microSD card breakout is also present on the main perfboard from an earlier prototyping stage, but the final firmware (Section 4.3) does not include any SPI/SD-card code, so it is currently unused.

Component placement on the main board was not constrained by a fixed grid or template; components were arranged experimentally, prioritizing short and uncrossed traces while still fitting the full board within the enclosure's footprint.

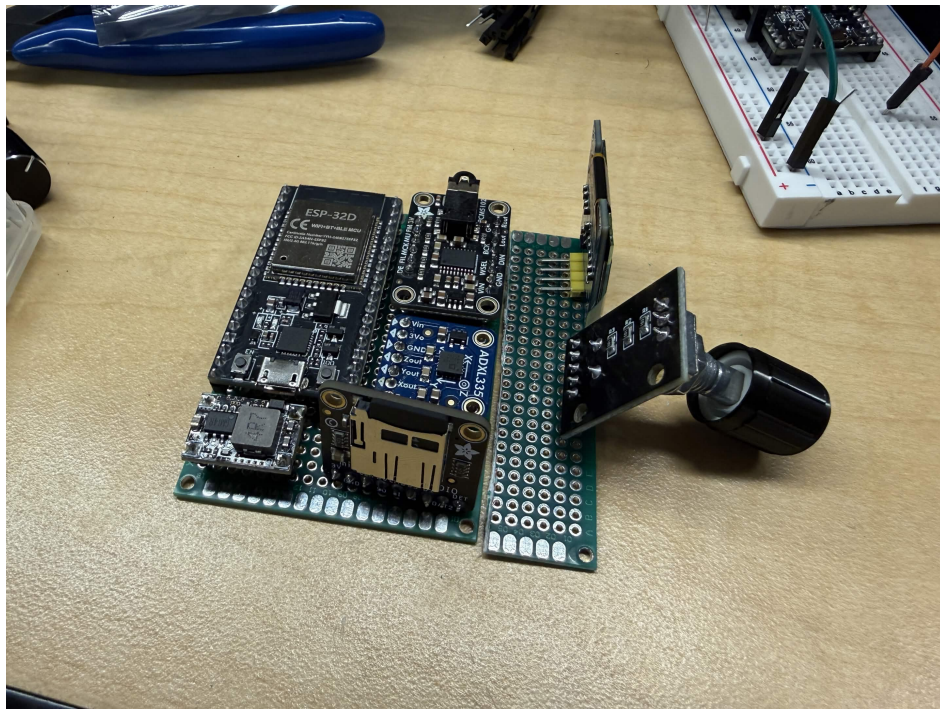


Figure 3: Perfboard assembly during construction, showing the ESP32, PCM5102 DAC, ADXL335 accelerometer, DC/DC converter, and microSD breakout (unused in the final firmware) on the main board, with the OLED/rotary-encoder board module still unattached at this stage

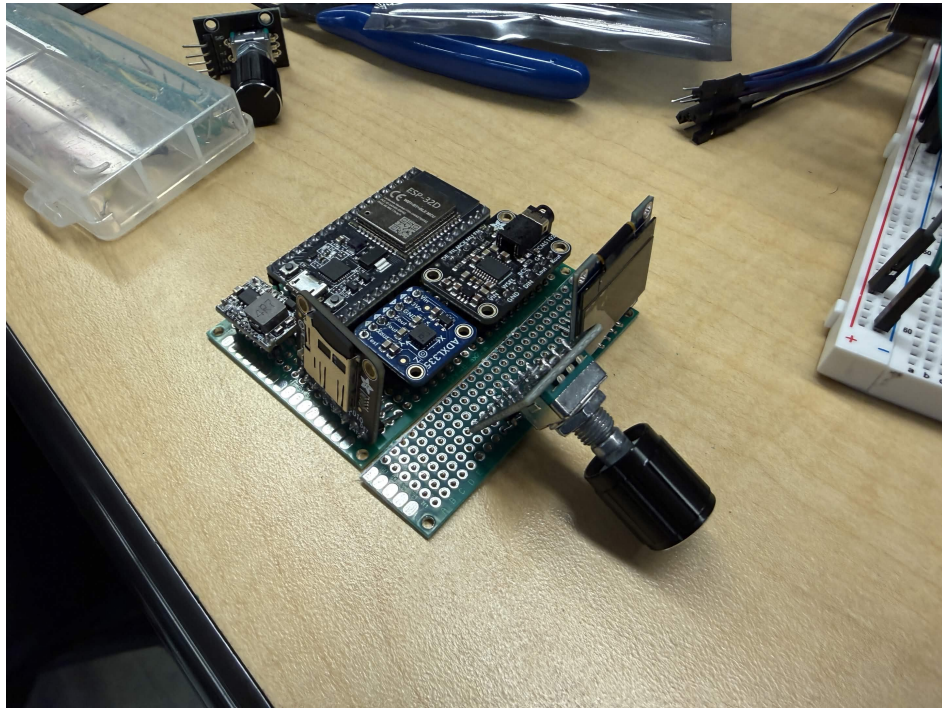


Figure 4: Main perfboard with the OLED display and rotary encoder board attached at a right angle to the main board

4.2.4 Power System

The system is powered by a 9V rechargeable battery feeding a DC/DC converter,¹ which supplies the ESP32's 5V input; the ESP32's own regulator then supplies the 3.3V rail used by every other subsystem (see Section 4.2 for the full power distribution).

The battery itself has a built-in USB-C charging port, which is exposed through a cutout in the enclosure so the whole unit can be charged without opening the case. The battery does not need to be removed for day-to-day charging; it only needs to come out if it fails, at which point the enclosure was designed so the battery sits snugly in place while the lid is closed but can still be dropped in and swapped easily when the lid is open (see Section 4.2.5).

¹Specific converter part number to be finalized.

The system’s average power draw is estimated at approximately 600 mW. This figure was derived from the combined peripherals operating at 3.3V drawing roughly 200–250 mA on average, and does *not* yet include the ESP32’s own current draw on the 5V rail while executing code – so actual total consumption is expected to be somewhat higher than 600 mW. A full per-component breakdown remains to be measured (Table VI). The perfboard also carries an unused microSD breakout (Section 4.2); since the final firmware never drives it, it is not expected to contribute meaningfully to system power draw.

Table VI: Power budget

Component	Current (mA)	Draw	Duty Cycle	Avg. (mW)	Power
Peripherals, combined (3.3V rail)	200–250		Continuous	~660–825	
ESP32 (5V rail, active)	30–60		Continuous	~130	

4.2.5 Mechanical and Enclosure Design

The enclosure was designed in Autodesk Fusion 360 and 3D-printed in PLA on a Bambu Lab A1 Mini printer available on campus. PLA was used simply because it was the material on hand; it was not selected against alternatives for any performance reason.

The design is an updated version of the enclosure used in the predecessor project, reusing its sliding-lid mechanism (rather than a screw-fastened lid) but adding several new cutouts specific to this system’s components: the rotary encoder knob, the OLED display, a power button, and the battery’s USB-C charging port are all accessible from outside the case without opening it. An additional cutout exposes the ESP32’s USB port so firmware can be reflashed while the

board remains mounted inside the enclosure, though this cutout is intended for development use rather than the end user.

The enclosure mounts to the body via a spring-loaded pocket-knife-style belt clip (purchased hardware, along with the mounting screws), which clips onto a waistband at the hip – consistent with the hip-mounted placement described in Section 4.2.



Figure 5: Completed 3D-printed enclosure, closed

4.2.6 Bill of Materials Summary

Table VII summarizes the top-level cost for a single unit; the full itemized bill of materials (including vendor and part number where applicable) is given in Appendix D.



Figure 6: Enclosure with sliding lid open, showing the battery seated in place

Table VII: Bill of materials summary

Part	Unit Cost	Qty	Line Total
ESP32	\$6.67	1	\$6.67
MAX30102 PPG Sensor	\$6.22	1	\$6.22
ADXL335 Accelerometer	\$4.43	1	\$4.43
Earbuds	\$10.00	1	\$10.00
9V Battery	\$5.00	1	\$5.00
Resistors	\$0.45	5	\$2.25
Capacitors	\$0.50	5	\$2.50
Enclosure (PLA, ~2 oz)	—	—	\$1.13
Assembly Hardware	\$7.00	1	\$7.00
Total			\$45.20

4.2.7 Design Revisions

I2C bus split. The first firmware revision used a single shared I2C bus (2 pins) for both the MAX30102 PPG sensor and the OLED display, addressing each device separately in software via its distinct hardware address. In principle this should work without conflicts, and the implementation was double-checked for blocking issues, but in practice one device – usually the OLED – would fail to initialize or stop responding to I2C commands. Since spare GPIO pins were available, the fix was to move to two independent I2C buses (4 pins total, one SDA/SCL pair per device) rather than continuing to debug the shared-bus approach. This fully resolved the initialization issue and introduced no downstream timing or blocking problems.

Enclosure. The enclosure went through several iterations to correct measurement inaccuracies (e.g., cutout alignment, internal component fit), but its overall floorplan and layout remained essentially unchanged from the first version.

Battery selection. Several battery packages and chemistries were considered before settling on the 9V rechargeable battery ultimately used. The team deliberately compared options rather than defaulting to the first choice, and stayed with the originally purchased battery throughout the rest of the project once it was confirmed to meet the system’s size, rechargeability, and power requirements.

Removal of microSD logging. An earlier prototype used a microSD card to log workout data, which could be reviewed after a session by connecting the card to a laptop and inspecting it in Microsoft Excel. This was primarily a debugging aid rather than a feature intended for the end user, and it was ultimately left out of the final firmware (Section 4.4). Since the microSD card was a non-negligible contributor to current draw, removing it also improves the system’s power margin

relative to earlier prototype measurements (Section 4.2.4).

4.3 Firmware and Embedded Software

4.3.1 MCU and Toolchain Overview

Firmware is written and flashed to the ESP32 using the Arduino IDE. Each peripheral – the MAX30102 PPG sensor, ADXL335 accelerometer, OLED display, PCM5102 DAC, and the ESP32 core itself – is supported by its own vendor/community Arduino library, installed through the IDE’s library manager.

Development followed a bottom-up approach: individual test programs were first written for each subsystem in isolation (one program each for the PPG, the accelerometer, the OLED, and so on) to understand and verify its behavior on its own. These were later refactored into a single non-blocking `.ino` file, where each subsystem’s logic became a function called from the main `loop()`, which acts as a scheduler coordinating all subsystems within one program.

4.3.2 Sensor Sampling and Data Pipeline

The main loop runs continuously with a 5 ms delay at the end of each iteration (so the effective loop period is roughly 5 ms plus whatever processing each function below takes – there is no fixed hardware sample-timer). Every iteration, the loop checks the encoder, samples the PPG sensor, and samples the accelerometer; the metronome and cadence controller only run once a workout has been started.

Heart-rate sampling. Each loop iteration reads a raw IR value from the MAX30102. If the IR value falls below a fixed finger-presence threshold, no finger is assumed to be on the sensor and the displayed BPM is cleared. Otherwise,

the IR sample is passed to the SparkFun MAX3010x library's `checkForBeat()` peak-detection function; when it reports a beat, the time since the previous beat is converted directly to instantaneous BPM ($60000/\text{interval}_{\text{ms}}$). Instantaneous BPM values are validated against a plausible range (60–220 BPM) before being accepted, and, while a workout is active, stored into a rolling 9-sample heart-rate buffer.

Cadence/step sampling. Each loop iteration also reads all three ADXL335 axes (each axis averaged over 5 raw ADC samples), converts the 12-bit readings to acceleration in g, and computes the 3-axis vector magnitude. Full step-detection logic is described in Section 4.3.3. Valid step-to-step intervals are converted to a steps-per-minute cadence value, validated against a plausible range (50–250 SPM), and, while a workout is active, stored into a rolling 9-sample cadence buffer alongside the heart-rate buffer above.

Both buffers are consumed and cleared every 10 seconds by the cadence controller (Section 4.3.3), which is the only place raw samples are turned into an updated feedback command.

4.3.3 Cadence Detection Algorithm

This is the system's core control algorithm: it turns a stream of raw heart-rate and step samples into an updated metronome tempo, with the goal of guiding the user's measured heart rate toward their chosen target.

Step detection. Rather than looking for peaks on a single axis, the algorithm computes the combined 3-axis acceleration magnitude, $\sqrt{x^2 + y^2 + z^2}$, on every loop iteration. Because the sensor's resting (1g) baseline drifts slightly with orientation and mounting, the algorithm tracks that baseline with a slow exponen-

tial moving average ($0.95 \times \text{old baseline} + 0.05 \times \text{current magnitude}$) rather than assuming a fixed constant. The filtered signal is the absolute difference between the current magnitude and this adaptive baseline. A step is registered when the filtered signal exceeds a fixed threshold ($0.35g$) *and* at least 250 ms (a cooldown period) has passed since the previous registered step – this cooldown acts as a debounce so a single footfall’s impact is not counted as multiple steps. The time between two consecutive registered steps is converted to a steps-per-minute value.

Heart-rate-to-cadence control. Every 10 seconds (`CONTROL_INTERVAL_MS`), the controller wakes up and checks whether at least 2 valid samples have accumulated in both the heart-rate and cadence buffers from the last 10 seconds. If not, the previously commanded cadence is simply held. Otherwise, it takes the median of each buffer (via an in-place sort) to get a noise-resistant heart-rate and cadence reading for that window, then computes a multiplicative correction ratio:

$$\text{ratio} = \frac{\text{targetHR} - \text{restingHR}}{\text{measuredHR} - \text{restingHR}}$$

This ratio expresses how far off the user’s actual heart-rate elevation (above resting) is from their desired elevation. Note that `restingHR` is currently a fixed constant (60 BPM) rather than a per-user calibrated value. If the measured heart rate is too close to this assumed resting heart rate (within 5 BPM) or the target is at or below resting – both of which would make the ratio unstable or meaningless – the update is skipped and the previous cadence is held instead. Otherwise, the new commanded cadence is the previous cadence used by the controller multiplied by this ratio, then clamped twice: first to the system’s absolute cadence range (50–250 SPM), and second to within ± 15 SPM of the current commanded cadence, so a single control update cannot swing the metronome tempo drastically. This

second clamp is the primary mechanism preventing oscillation or jarring jumps in the audio feedback. A simplified version of this controller is shown in Listing 4.1.

Figure 7: Cadence detection and heart-rate-to-cadence control flow

Listing 4.1: Heart-rate-to-cadence controller (simplified from the final firmware)

```
1 // Runs once every CONTROL_INTERVAL_MS (10 s), given >=2
   valid samples
2 // in both the heart-rate and cadence buffers over that
   window:
3 float measuredHR      = median(heartRateBuffer);
4 float measuredCadence = median(cadenceBuffer);
5
6 float numerator      = targetHR - restingHR;
7 float denominator = measuredHR - restingHR;
8
9 if (denominator <= 5.0f || numerator <= 0.0f) {
10     // Unsafe/unstable ratio -- keep previous commanded
       cadence.
11 } else {
12     float ratio = numerator / denominator;
13     float proposed = cadenceUsedByController * ratio;
14
15     proposed = clamp(proposed, MIN_CADENCE, MAX_CADENCE);
16     proposed = clamp(proposed,
17                       commandedCadence - MAX_CADENCE_CHANGE,
18                       commandedCadence + MAX_CADENCE_CHANGE)
       ;
```

```
19  
20     commandedCadence = proposed;  
21     setMetronomeCadence(commandedCadence);  
22 }
```

4.3.4 Feedback Trigger Logic

The user selects a single target heart-rate value (not a range) via the rotary encoder and OLED display before starting a workout (Section 4.4). The commanded cadence produced by the controller in Section 4.3.3 is converted directly to a metronome beep interval ($60000/\text{cadence}_{\text{SPM}}$ ms), and a short tone is played through the earbuds each time that interval elapses. The user is instructed to match their running cadence (footfalls) to this metronome rhythm, using it as a pacing cue to naturally raise or lower their heart rate toward the target.

Rather than thresholds and hysteresis around a target zone, the system’s anti-oscillation behavior comes from the ± 15 SPM per-update clamp described above: because the commanded cadence can only change gradually every 10 seconds, the audible tempo changes smoothly rather than jumping or flickering in response to noisy instantaneous readings.

4.3.5 Communication Protocol

Although the ESP32 supports Wi-Fi and Bluetooth, this design does not use any wireless communication link. There is no companion mobile app and no data is transmitted off-device wirelessly; all user interaction is local, through the rotary encoder and OLED display, and feedback is delivered locally through the wired earbuds (Section 4.4).

4.3.6 Power Management Firmware

The firmware does not implement any active power management: there is no sleep/low-power state for the ESP32 or sensors, and no dynamic adjustment of sampling rate. The MAX30102 PPG sensor does support a low-power mode, but it is not used in the current firmware, since the system already meets its battery-life requirement (Section 4.2.4) without it.

This was a deliberate scope decision rather than an oversight: while the device is powered on, it is assumed to be actively in use, so there is limited benefit to power-saving behavior during a workout. The one scenario this does not address is a user turning the device on and forgetting to turn it off afterward; an auto-shutoff after a period of inactivity, or enabling the PPG's low-power mode between samples, would be natural additions if this becomes a real usage problem.

4.4 Application and User-Facing Software

4.4.1 Architecture Overview

There is no separate mobile app or desktop dashboard. All user-facing "application" logic runs entirely on-device, through the OLED display and rotary encoder described in Section 4.2: the OLED shows setup and in-workout screens, and the rotary encoder (turn to adjust a value, push to confirm/start/stop) is the sole input mechanism.

4.4.2 Feedback Modalities

Audio is the only feedback modality: a metronome-style tone played through the wired earbuds, whose tempo is set by the cadence controller (Section 4.3.3).

The OLED display is informational rather than a feedback channel in its own right – it shows the target heart rate, current heart rate, measured cadence, and commanded cadence during a workout, but does not itself guide the user’s pacing the way the audio does. There is no haptic feedback.

4.4.3 Data Visualization and Logging

The current firmware includes no data logging. Earlier in development, a microSD card was used to log workout data, which could then be pulled out, connected to a laptop via its SD port, and reviewed in Microsoft Excel – primarily as a debugging aid rather than an end-user feature. This was ultimately left out of the final firmware (Section 4.3) since it was not necessary for the end-user experience, though the SD breakout itself remains physically present, unused, on the perfboard (Section 4.2).

4.4.4 UX Design Decisions

The on-device UI went through several iterations:

1. An initial version let the user select a single desired heart rate and a workout duration via the rotary encoder. Debounce logic for the encoder was implemented from this first version onward to keep the interface usable.
2. This was expanded to support two desired heart-rate/duration pairs, along with a visible, continuously updating on-screen timer during a workout.
3. User age was then added as an input, to estimate maximum attainable heart rate using the standard "220 minus age" rule.

4. Building on the age-based maximum, desired heart rate was briefly reworked from a raw numeric entry into a selection of named target workout zones (e.g., fat-burn, cardio, peak), computed as ranges relative to that estimated maximum.

The final firmware (Listing 4.1 and Section 4.3.3) has settled back on a single, direct numeric target heart-rate value (adjustable from 60–200 BPM via the encoder) rather than the named-zone selector, matching the interface actually shipped in the current code.

Chapter 5

Test Plans

5.1 Test Methodology

Testing was performed by both team members, though the majority of the on-body cadence and heart-rate data reported in this document comes from one partner acting as the primary test subject. There was no fixed number of trials specified up front; instead, the team ran 2–3 test sessions after each significant firmware change, each session consisting of a 1–2 minute run, followed by reviewing the collected sensor data. If behavior looked correct, the change was considered verified; if not, 1–2 additional runs were repeated to confirm whether the issue was consistent or a one-off. Test runs took place in a mix of environments: indoors around the EE building’s hallways, and outdoors around campus, including both uphill and downhill sections, to expose the system to a range of grades and motion patterns rather than flat ground alone.

For heart-rate verification specifically, ground truth was established manually: the test subject counted their own pulse by feel, placing two fingers on the carotid artery in the neck, rather than using a second reference heart-rate device.

5.2 Test Matrix

Table VIII restates the verification method planned for each requirement from Chapter 3, without the pass/fail outcome – those results, along with the full test data, are presented in Chapter 7 once testing is complete.

Table VIII: Planned verification method by requirement

ID	Design Feature	Planned Verification Method
REQ-1	MAX30102 PPG sensor interfaced to the ESP32 via I2C	Compare measured BPM against manual carotid pulse count
REQ-2	ADXL335 accelerometer analog output sampled by ESP32 ADC	Bench tilt/shake testing, followed by on-body step-detection testing during a run
REQ-3	Rotary encoder and OLED display for target heart-rate selection	Functional test of knob input and menu navigation
REQ-4	ESP32 firmware combining PPG and accelerometer data into a pacing signal	End-to-end on-body test of pacing output against a chosen target heart rate
REQ-5	PCM5102 DAC driving earbuds with an audible rhythm	Audio output and audibility testing
REQ-6	9V rechargeable Li-ion battery with DC/DC conversion powering all components	Power budget calculation from measured current draw
NFR-1 (Latency)	Non-blocking real-time firmware loop	Measure time from heart-rate change to audio update

ID	Design Feature	Planned Verification Method
NFR-2 (Battery life)	9V, 1200 mAh Li-ion battery with DC/DC converter	Calculate from measured current draw and converter efficiency
NFR-3 (Weight)	Perfboard assembly with 3D-printed PLA enclosure	Physically weigh components and enclosure
NFR-4 (Cost)	Consumer-grade components (ESP32, MAX30102, ADXL335, etc.)	Bill-of-materials calculation for a single unit
NFR-5 (Accuracy)	MAX30102 signal filtering and beat-detection logic	Compare against manual carotid pulse count across multiple on-body test runs

Chapter 6

Development and Construction

6.1 Bench and Unit Testing

Before full system integration, each core sensor was tested in isolation on the bench.

MAX30102 (PPG). The primary bench test compared the sensor's BPM output against a manual carotid pulse count, both taken over a one-minute window: one team member recorded their own pulse manually for one minute as a baseline, then recorded the PPG sensor's BPM output over the same one-minute window, immediately after. This was repeated several times both at rest (stationary bench testing) and during exercise. Across these tests, the sensor's mean absolute error against the manual baseline was approximately 4.2 BPM.¹

ADXL335 (accelerometer). Testing was primarily functional rather than quantitative: each axis's output voltage was observed while tilting the sensor, using an oscilloscope, and code was written to convert raw voltage to an approxi-

¹This figure is from memory pending confirmation of the most up-to-date recorded data from the team member who ran most of the testing.

mate g-force value. The absolute accuracy of that g-force conversion was not the focus, since the cadence-detection algorithm (Section 4.3.3) only relies on relative *changes* in acceleration to detect steps, not absolute, calibrated accelerometer readings.

Power. A bench-level current/power measurement was also taken to sanity-check the estimated 600 mW average power draw (Section 4.2.4); the measured value was close enough to that estimate to consider it validated at the bench level, ahead of the full battery-life testing described in Chapter 7.

Table IX: Bench test results

Test	Measured	Target	Pass/Fail
MAX30102 BPM vs. manual pulse count	~4.2 BPM MAE	N/A (accuracy check)	Pass
ADXL335 axis response to tilt	Functional (relative change only)	Detects change in orientation	Pass
Bench power draw	~600 mW (matches estimate)	~600 mW estimated	Pass

6.2 Team Roles and Division of Labor

Work was split between divide-and-conquer tasks and tasks done jointly. Individual component bench testing – such as characterizing the ADXL335 and MAX30102 in isolation, described above – was generally done independently by whichever team member picked up that component, while construction of the physical prototype and higher-level decisions (component selection, system layout, firmware architecture) were made together as a team.

In terms of primary ownership, one team member led the mechanical and enclosure design (CAD modeling and 3D printing, Section 4.2.5) along with the

initial firmware architecture, while the other led later firmware refinement and final system integration (including the pin reassignments and the removal of microSD logging described in Section 4.2). This reflects a difference in *area of ownership* rather than total effort – overall, the two team members contributed roughly evenly to the project as a whole.

Code was shared and version-controlled through a GitHub repository, and other project resources (datasheets, documents, test data) were kept in a shared Google Drive.

6.3 Risks Encountered and Mitigations

Several design-level revisions already described in Chapter 4 – the shift to two independent I2C buses (Section 4.3), multiple enclosure iterations (Section 4.2.5), the battery selection process, and the removal of microSD logging – were themselves responses to risks encountered during development, and are not repeated here.

One additional risk arose from an early component choice: the PPG sensor originally ordered was an ADPD144RI, but its physical package turned out to be essentially unusable for this design (e.g., not practical to hand-solder/integrate into the perfboard assembly at the finger). The team switched to the MAX30102, which is used throughout the rest of this report, but this substitution introduced a short schedule delay while waiting for the replacement part to arrive.

Chapter 7

Integration and Test Results

7.1 System Integration

7.1.1 Hardware-Firmware-App Integration Process

Both hardware and firmware were developed and integrated bottom-up. On the hardware side, each sensor and peripheral was first brought up and verified on its own (Section 6), then wired together on a breadboard and tested with both a laptop and the battery powering the system, before finally being soldered onto the perfboard assembly used in the completed device (Section 4.2). Firmware followed the same pattern: an individual test program was written for each subsystem in isolation, and these were later refactored into the single non-blocking main loop that ships in the final firmware (Section 4.3). Because there is no separate mobile or desktop application (Section 4.4), integration in practice meant combining the hardware sensor/actuator subsystems with this single firmware loop, rather than integrating a separate software layer.

7.1.2 Integration Challenges and Resolutions

The most significant integration-level issue, beyond the I2C bus contention already described in Section 4.2, surfaced once the individually-tested subsystems were combined into the single main loop: the MAX30102’s internal buffer would stop clearing properly, a symptom that had not appeared during standalone PPG testing. This turned out to be a blocking issue caused by another subroutine in the loop – specifically, the OLED display update was interfering with the PPG read when called in a particular order. The fix was to reorder the main loop so that the heart-rate update (`updateHeartRate()`) runs *before* the OLED update (`updateOLED()`) on each iteration, rather than after; this resolved the buffering issue with no other side effects observed.

7.2 On-Body Testing Results

Cadence accuracy. The device’s measured cadence (steps per minute) was compared against ground-truth footfalls counted during on-body test runs. Most of this testing and data collection was performed by one team member. Across these trials, the device’s cadence readings had a mean absolute error of 4.75, corresponding to approximately 87.4% accuracy.

Feedback response time. The approximately 1-second latency from a heart-rate change to a corresponding audio feedback change (Section 3.2) was consistent between resting and exercising on-body tests – it was not found to depend on whether the user was stationary or actively running.

Comfort. No formal user comfort survey with a meaningful sample size was conducted; comfort was assessed informally by the two team members wearing the device themselves. Both found it subjectively more comfortable than the

predecessor design (Section 4.2), consistent with the motivation for moving away from the chest strap and in-shoe FSR. Some residual discomfort remains, as would be expected of any wearable device, but the team’s subjective assessment is that comfort was meaningfully improved relative to the predecessor. This falls short of the $\geq 90\%$ formal approval-survey target stated in Chapter 3, which remains an open verification item rather than a completed one.

Figure 8: Measured vs. ground-truth cadence across trials

7.3 Requirements Verification

Table III in Chapter 3 presents the full requirements traceability matrix, including the verification method and final result for every functional and non-functional requirement; it is not repeated in full here. In summary: all six functional requirements (REQ-1 through REQ-6) and four of the five non-functional requirements (latency, battery life, weight, and cost) passed against their stated targets. The accuracy requirement (NFR-5) was measured as a ~ 4.2 BPM mean absolute error, which corresponds to roughly 93–98% accuracy depending on where in the device’s 60–200 BPM operating range a reading falls (Chapter 3) – meeting the $95\% \pm 5\%$ target at higher heart rates, but falling slightly short of it at the low end. This is discussed further in Section 7.5. The comfort requirement (NFR-6, $\geq 90\%$ user approval) remains unverified rather than passed or failed: no formal comfort survey was conducted (Section 7.1), so only an informal, two-person assessment is available.

7.4 Results and Discussion

7.4.1 Summary of Key Results

Table X collects the key quantitative results from testing, alongside their targets.

Table X: Summary of key results against targets

Metric		Target	Achieved
Heart-rate accuracy (MAE)		95% $\pm$ 5% (equivalent)	$\sim$ 4.2 BPM MAE ($\sim$ 93–98% depending on HR)
Feedback latency		$\leq$ 5 s	$\sim$ 1 s (resting and exercising)
Battery life		6 hr	$\sim$ 5.8 hr (calculated)
Weight		12 oz (16 oz max)	$\sim$ 10 oz
Cost (single unit)		\$175	\$45.20
Comfort		$\geq$ 90% approval (survey)	Not formally measured; subjectively improved over predecessor
Cadence accuracy		–	4.75 / 87.4%

7.4.2 Interpretation

Relative to the project’s original objective – translating physiological measurements into simple, actionable pacing guidance (Chapter 1) – the results are largely positive. Feedback latency of approximately 1 second is well within the 5-second target, supporting the goal of real-time guidance rather than delayed, after-the-fact feedback. Heart-rate accuracy, expressed as a $\sim$ 4.2 BPM mean absolute error, translates to roughly 93–98% accuracy depending on where in the operating range a reading falls – meeting the 95% target at the higher end of the range, but falling a bit short at lower heart rates. In practice this means the system’s cadence feed-

back should still track a user’s actual cardiovascular state reasonably well, though with more noise than originally hoped, particularly for users whose target heart rate sits toward the lower end of the supported range. The battery life, weight, and cost results all comfortably meet their targets, supporting the practical wearability and affordability goals that motivated moving away from the predecessor’s chest-strap-based design (Chapter 4). The two open items – formal comfort verification and cadence accuracy – mean the system’s core promise (comfortable, accurate real-time pacing guidance) is well-supported by the latency, cost, and heart-rate results, but is not yet fully verified end-to-end.

7.4.3 Comparison to Existing Products and Prior Art

Table I in Chapter 2 compares this project against the predecessor design and several commercial products. Unlike Garmin Running Dynamics and Polar’s wrist-based training products, which require the user to glance at a screen to interpret their heart-rate status, and unlike WHOOP’s retrospective, post-workout coaching model, this project’s single-earbud auditory cadence feedback provides guidance continuously and in real time without requiring visual attention – directly supporting the usability motivation described in Chapter 1. A direct quantitative accuracy comparison against these commercial products was not possible, since their heart-rate and cadence accuracy figures are proprietary and not publicly benchmarked; the comparison here is therefore limited to sensing approach and feedback modality rather than head-to-head accuracy.

7.4.4 Unexpected Findings and Failure Modes

The two most significant unexpected issues encountered during development were both integration-level rather than component-level: the I2C bus contention between the MAX30102 and OLED display when sharing a single bus (Section 4.2), and the MAX30102 buffer failing to clear due to a loop-ordering dependency with the OLED update (Section 7.1). Neither issue was apparent when the affected components were tested individually – both only surfaced once the full system was integrated, which is itself a useful finding: bench testing of the MAX30102 and OLED in isolation did not exercise the interaction between them that ultimately caused problems.

7.5 Limitations of Testing

Testing throughout this project has several limitations that should be kept in mind when interpreting the results above:

- **Small test population.** All on-body testing was performed by the two team members themselves (effectively $n = 2$), rather than a broader or more diverse set of test subjects. Results may not generalize to users with different body types, gait patterns, or skin tones (which can affect PPG signal quality).
- **Ground truth measurement error.** Heart-rate ground truth was established by manual carotid pulse counting (Section 7.1) rather than a validated reference heart-rate monitor, which introduces its own counting error into the reported ~ 4.2 BPM MAE figure (and the 93–98% accuracy range derived from it).

- **Controlled testing conditions.** Test runs, while varied between indoor hallways and outdoor inclines (Section 5), were still short (1–2 minutes) and conducted on or near campus, rather than during longer, more varied real-world workouts.
- **Incomplete data.** Comfort-approval target was never formally measured with a meaningful sample size.

Chapter 8

Conclusion and Future Work

8.1 Summary of Contributions

This project delivered a working, wearable, cadence-based heart-rate feedback device that improves on its predecessor (Chapter 4) in several concrete ways: replacing a chest-strap heart-rate monitor and shoe-mounted force-sensitive resistor with a finger-worn PPG sensor and a hip-mounted accelerometer, consolidating nearly all electronics into a single enclosure, and delivering real-time pacing guidance through a simple audio metronome rather than a screen the user must glance at. At the core of the system is a multiplicative heart-rate-to-cadence controller (Section 4.3.3) that periodically re-targets the metronome’s tempo based on how far the user’s measured heart rate is from their chosen target, with rate limiting to keep the audio feedback smooth rather than jumpy.

Against the project’s original requirements (Chapter 3), the completed system meets its cost (\$45.20 vs. a \$175 target), weight (~10 oz vs. a 12 oz target), battery life (~5.8 hr calculated vs. a 6 hr target), and latency (~1 s vs. a 5 s target) requirements comfortably, and comes close on heart-rate accuracy (~4.2 BPM

MAE, equivalent to roughly 93–98% depending on heart rate, vs. a $95\% \pm 5\%$ target). Two verification items – cadence accuracy and formal comfort approval – remain open (Section 7.4), but the system as built and tested represents a functional, meaningfully improved successor to the predecessor design that motivated this project.

8.2 Lessons Learned

Several of the project’s more valuable lessons came from problems that only appeared once individually-tested pieces were combined, rather than from any single component in isolation. The I2C bus contention between the MAX30102 and OLED display, and the later PPG-buffer-clearing issue traced to main-loop ordering (Section 7.1), both passed standalone component testing without issue and only surfaced during full-system integration – underscoring that bench-testing a component alone does not guarantee it will behave the same way once sharing a microcontroller and timing budget with everything else.

Component selection also carried real schedule risk: the originally ordered ADPD144RI PPG sensor turned out to have an impractical physical package, and switching to the MAX30102 cost real development time while the replacement part was ordered and shipped (Section 6), a reminder to validate a part’s physical/mechanical fit early, not just its electrical specifications.

On the process side, developing both hardware and firmware bottom-up – verifying each subsystem alone before integrating it (Section 7.1) – made the eventual integration bugs easier to isolate and fix, even though it did not prevent them entirely. Splitting ownership by area (mechanical/enclosure and initial firmware versus firmware refinement and final integration, Section 6) while sharing code

through GitHub and resources through a shared Google Drive worked well for a two-person team. Finally, deliberately cutting scope – removing the microSD logging feature once it was clear it was a debugging aid rather than something the end user needed (Section 4.4) – simplified the firmware and improved the power budget, a useful reminder that not every feature built during development needs to ship in the final design.

8.3 Future Work

A few directions stand out for extending this project:

- **Broader, formal user testing.** All testing to date has been performed by the two-person development team (Section 7.5). A larger and more diverse study – different body types, skin tones, and gait patterns – would give a much stronger picture of real-world accuracy and comfort, and would finally provide the formal comfort-approval data the original requirements called for but this project did not collect.
- **Closing the accuracy gap.** Heart-rate accuracy (~ 4.2 BPM MAE, ~ 93 – 98% depending on heart rate) fell just short of the 95% target at lower heart rates. Further refinement of the PPG filtering algorithm, or exploring the accelerometer-based motion-artifact rejection approach considered during development, could help close this gap.
- **Power management.** The current firmware implements no active power management (Section 4.3); adding an auto-shutoff after a period of inactivity, or enabling the MAX30102’s low-power mode between samples, would guard against battery drain from a device left on unintentionally.

- **Battery protection circuitry.** Dedicated battery protection circuitry, identified as a design goal but not implemented in the current prototype (Chapter 3, Health and Safety), would be a natural next step toward a safer, more production-ready design.
- **Revisiting deferred UX features.** Several interface ideas explored during development – age-based maximum heart-rate estimation and named target-zone selection (Section 4.4) – were ultimately not part of the shipped firmware. With the core sensing and control system now validated, these could be revisited as usability enhancements.
- **Moving beyond perfboard.** The current build is entirely perfboard-based (Section 4.2). A custom PCB would allow for a smaller, more robust, and more manufacturable final product. The apple watch could also be a substitute for the PPG and accelerometer and the software can be turned into an app.

Bibliography

- [1] A. F. Machado *et al.*, “High-intensity interval training using whole-body exercises: Training recommendations and methodological overview,” *Clinical Physiology and Functional Imaging*, vol. 39, no. 6, pp. 378–383, 2019.
- [2] A. Nguyen *et al.*, “A randomized pilot study comparing the impact of strengthening-based running training with only running on the incidence of running-related injuries among novice runners,” *Sports (Basel, Switzerland)*, vol. 12, no. 1, p. 25, 2024.
- [3] A. Nazaret *et al.*, “Modeling heart rate response to exercise with wearables data.” Apple, 2022.
- [4] J. Ni, L. Muhlstein, and J. McAuley, “Modeling heart rate and activity data for personalized fitness recommendation,” in *The World Wide Web Conference (WWW '19)*, (New York, NY, USA), pp. 1343–1353, Association for Computing Machinery, 2019.
- [5] L. Rash and C. Rodriguez, “Cardio trainer,” Master’s thesis, Department of Electrical Engineering, California Polytechnic State University, San Luis Obispo, USA, 2023. B.S. thesis.
- [6] T. M. Cheng, A. V. Savkin, B. G. Celler, S. W. Su, and L. Wang, “Nonlinear

- modeling and control of human heart rate response during exercise with various work load intensities,” *IEEE Transactions on Biomedical Engineering*, vol. 55, no. 11, pp. 2499–2508, 2008.
- [7] A. Elliott, “Nonlinear integral control schemes for a cadence-heartrate process: A matlab exploration,” Master’s thesis, EE Dept., California Polytechnic State University, San Luis Obispo, USA, 2025.
- [8] D. R. Seshadri, R. T. Li, J. E. Voos, J. R. Rowbottom, C. M. Alfes, C. A. Zorman, and C. K. Drummond, “Wearable sensors for monitoring the internal and external workload of the athlete,” *NPJ Digital Medicine*, 7 2019.
- [9] L. Rash, “Providing cadence feedback in real-time to guide cardiovascular workouts,” Master’s thesis, EE Dept., California Polytechnic State University, San Luis Obispo, USA, 2024.
- [10] Garmin Ltd., “Running dynamics.” Garmin Owner’s Manual, 2026. Accessed 2026-07-20.
- [11] Polar Electro Oy, “The what and how of polar’s wrist-based heart rate measurement.” Polar Support, 2026. Accessed 2026-07-20.
- [12] WHOOP, Inc., “Whoop heart rate algorithm — accuracy, testing, updates.” WHOOP, 2026. Accessed 2026-07-20.
- [13] U.S. Department of Labor, “Lithium-ion batteries: Supply chain and child labor risks.” Bureau of International Labor Affairs. Accessed 2026-07-20.
- [14] L. Piwek, D. A. Ellis, S. Andrews, and A. Joinson, “The rise of consumer health wearables: Promises and barriers,” *PLOS Medicine*, vol. 13, no. 2, 2016.

-
- [15] J. M. Jakicic *et al.*, “Effect of wearable technology combined with a lifestyle intervention on long-term weight loss: The idea randomized clinical trial,” *JAMA*, vol. 316, no. 11, 2016.
- [16] Espressif Systems Co., Ltd., “Esp32 series datasheet,” tech. rep., Espressif Systems Co., Ltd., Shanghai, China, 2025. Version 5.2.

Appendices

Appendix A

Analysis of Senior Project Design

Project Title: A Cadence-Based Bio-Feedback System for Workout Optimization

Student's Name: Ronnie Sidhu

Student's Signature:

Advisor's Name: Dr. Vladimir Prodanov

Advisor's Initials:

Date:

Summary of Functional Requirements

This project is a wearable device that helps a user reach and hold a target heart rate during exercise. It measures the user's heart rate with a finger-worn sensor and their step cadence with a hip-mounted motion sensor, then plays an audible metronome tone through earbuds whose tempo the user matches with their footfalls. The device periodically adjusts that tempo based on how the user's measured heart rate compares to the target they selected before starting, guiding them to speed up or slow down without requiring them to look at a screen. The whole system runs self-contained on a rechargeable battery, with a knob-and-display interface used only to set the target heart rate and start or stop a

session.

Primary Constraints

The most significant constraint was the choice of ground truth for heart-rate accuracy: rather than a validated reference heart-rate monitor, the team used manual pulse counting (counting one’s own pulse by feel) as the baseline, since it was the most accessible method available – not necessarily the most accurate. This baseline was effectively treated as 100% correct for accuracy comparisons, which limits how much confidence can be placed in the resulting accuracy figures. Relatedly, all on-body data collected – both for heart-rate accuracy and cadence – comes from a sample size of two (the project’s own team members). While the results look promising, a larger and more diversified test population would be needed to strengthen these conclusions. Beyond testing methodology, the fixed 15-week project timeline was itself a constraint that shaped scope decisions throughout (e.g., the decision not to pursue formal user comfort surveys or a custom PCB, described in Chapters 4 and 8).

Economic

- **Original estimated cost of component parts:** No formal estimate was recorded at the start of the project. In hindsight, an informal guess at that time would likely have been around \$100.
- **Actual final cost of component parts:** \$45.20 (see Table VII in Chapter 4).

- **Final bill of materials:** see Appendix D.
- **Additional equipment costs:** None. The 3D printer used for the enclosure was accessed for free through the campus Makerspace, and no additional tools beyond a screwdriver (already on hand) were purchased for development.
- **Original estimated development time:** No separate time estimate was made beyond the fixed 15-week course timeline (EE460/463/464).
- **Actual development time:** Approximately the full 15-week timeline was used.
- **If manufactured on a commercial basis** (see Chapter 3, Design Constraints, for the full derivation):
 - Estimated number of devices sold per year: 1,000,000
 - Estimated manufacturing cost per device: \$25
 - Estimated purchase price per device: \$175
 - Estimated profit per year: approximately \$150,000,000 (gross)
 - Estimated cost for the user to operate the device: the device runs on a rechargeable battery with no per-use consumable cost beyond ordinary electricity for charging.

Environmental

The device's primary environmental impact stems from manufacturing and end-of-life disposal rather than normal operation, which produces no emissions. Lithium-ion battery production requires mining of lithium, cobalt, and rare earth elements,

associated with habitat destruction and pollution in extraction regions [13], and semiconductor fabrication for the microcontroller and sensors similarly contributes industrial waste and energy consumption upstream of assembly. The device’s main environmental burden occurs at end of life, when it becomes electronic waste.

Manufacturability

The device relies on commercially available electronic components that benefit from established consumer-electronics manufacturing infrastructure, supporting scalable production. Global semiconductor supply constraints could delay sourcing or increase costs, however. At commercial scale, manufacturing would also require replacing the prototype’s 3D-printed enclosure with an injection-molded one and adding a formal final-assembly process.

Sustainability

Using a rechargeable battery, as this design does, reduces disposable battery waste relative to a single-use alternative, though it introduces lithium battery handling considerations (see Health and Safety, below). Designing for component-level replacement – most notably the battery, which is already easily swappable in the current enclosure (Chapter 4) – could extend the product’s usable life and reduce electronic waste at the cost of additional design complexity.

Ethical

Some materials used in electronic components and batteries are sourced from regions where labor conditions may be unsafe or poorly regulated; the U.S. Department of Labor has identified lithium-ion battery supply chains specifically as an area of child labor and unsafe working condition risk [13]. A commercial version of this product would need to prioritize suppliers who follow ethical labor practices and internationally recognized safety standards.

Health and Safety

Lithium-based batteries are the primary in-device safety concern, as they can become hazardous if damaged, overheated, or improperly charged. Dedicated battery protection circuitry was identified as a design goal but is not implemented in the current prototype beyond a physical on/off switch, and remains an area for future improvement (Chapter 8). During use, audio output levels should also remain within safe listening limits so the device does not pose a hearing-safety risk.

Social and Political

Stakeholders include the project’s engineers, potential manufacturers and investors, and the end users – beginner runners and recreational athletes – as well as academic institutions involved in the technology’s development. Wearable, real-time physiological feedback has been associated with improved exercise adherence and cardiovascular training outcomes [14, 15], suggesting a plausible public-health benefit if adopted more broadly. Production would also depend on global electronics

supply chains spanning regions with varying labor and environmental standards, raising the same ethical sourcing concerns discussed above.

Development

Two things stood out as genuinely new, independently-learned skills during this project. First, developing firmware of this scale (a full sensor-fusion and control loop running on a single microcontroller) from the bottom up – individually testing each subsystem before integrating them into one program (Chapter 4) – was a new approach at this scale of complexity. Second, learning Autodesk Fusion 360 well enough to design the project’s enclosure was picked up independently in about one to two hours, specifically for this project.

Engineering Standards

This project does not target certification against a specific communications or safety standard (Chapter 2, Relevant Standards). The final design has no wireless communication, so standards such as FCC Part 15 or Bluetooth SIG certification do not apply. No dedicated battery protection standard was implemented beyond following the battery manufacturer’s own charging and handling recommendations.

Appendix B

Full Schematics

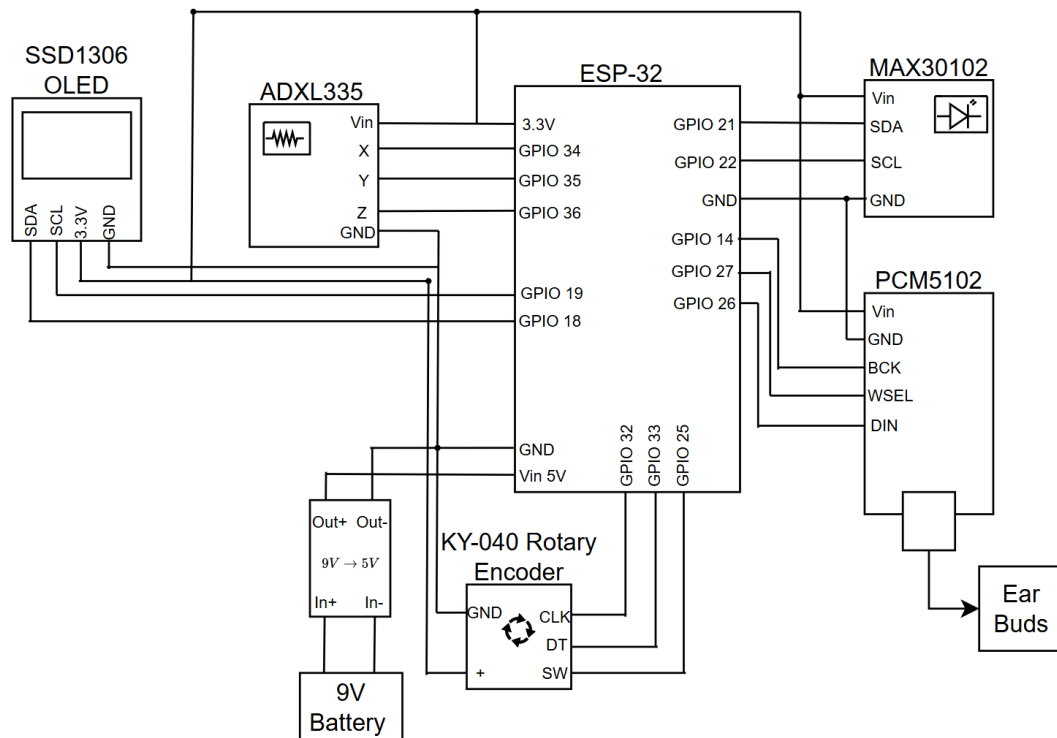


Figure 7: Final Schematic

Appendix C

Perfboard Layout Images

Figures 8 and 9 show the completed main and UI perfboards; see Section 4.2 for a description of the layout.

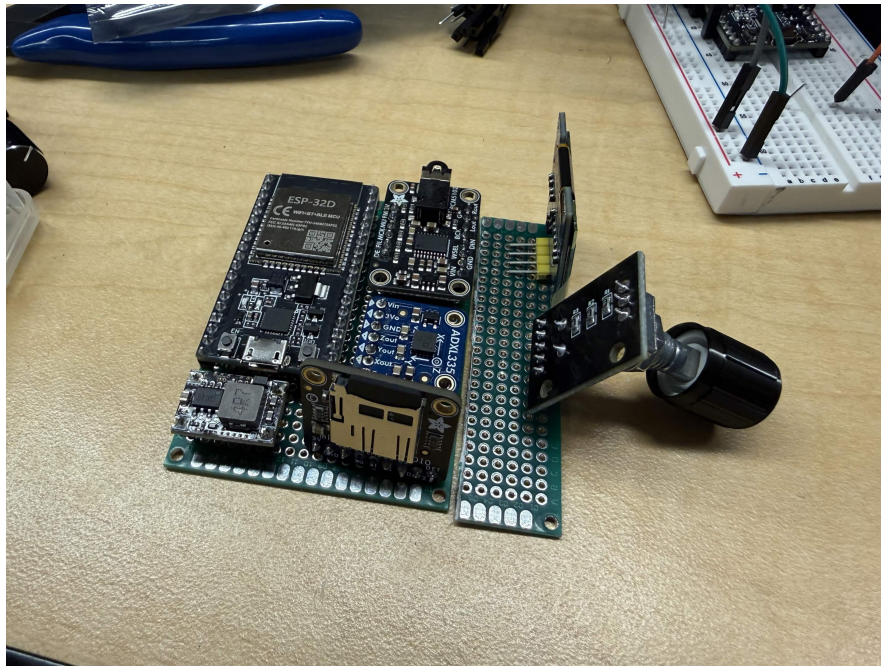


Figure 8: Main perfboard assembly

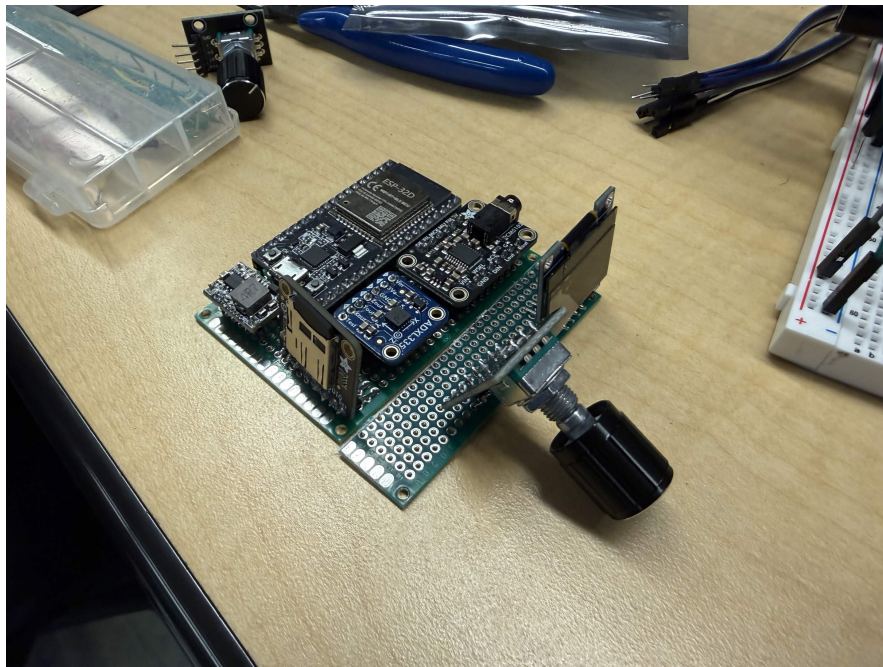


Figure 9: Main perfboard with UI (OLED/rotary encoder) board attached

Appendix D

Full Bill of Materials

Table XI: Complete bill of materials

Part	Manufacturer	Qty	Unit Cost	Line Total
/ Vendor				
ESP32	—	1	\$6.67	\$6.67
MAX30102	PPG —	1	\$6.22	\$6.22
Sensor				
ADXL335	Ac- —	1	\$4.43	\$4.43
celerometer				
Earbuds	—	1	\$10.00	\$10.00
9V Battery	—	1	\$5.00	\$5.00
Resistors	—	5	\$0.45	\$2.25
Capacitors	—	5	\$0.50	\$2.50
Enclosure (PLA,	—	—	—	\$1.13
~2 oz, 3D-printed				
in-house)				

Part		Manufacturer	Qty	Unit Cost	Line Total
		/ Vendor			
Assembly	Hard-	–	1	\$7.00	\$7.00
ware	(connectors,				
wiring,	mounting,				
belt clip)					
Total					\$45.20

Appendix E

Firmware Source Code

The complete final firmware is hosted on GitHub: https://github.com/ronnie-s2/Cadence_Feedback/blob/main/Cadence_Based_Feedback_System.txt. The two functions most central to this project's technical contribution – step/cadence detection and the heart-rate-to-cadence controller (Chapter 4, Section 4.3.3) – are reproduced below; see the GitHub link above for the full source, including sensor setup, the OLED interface, and the I2S audio driver.

Step/Cadence Detection

```
1 void updateStepDetection(unsigned long now) {  
2     int xRaw = readAveragedAxis(ADXL_X);  
3     int yRaw = readAveragedAxis(ADXL_Y);  
4     int zRaw = readAveragedAxis(ADXL_Z);  
5  
6     float xAccel =  
7         map(xRaw, ADC_MIN, ADC_MAX, -3000, 3000) / 1000.0f;  
8     float yAccel =
```

```
9      map(yRaw, ADC_MIN, ADC_MAX, -3000, 3000) / 1000.0f;
10  float zAccel =
11      map(zRaw, ADC_MIN, ADC_MAX, -3000, 3000) / 1000.0f;
12
13  float accelerationMagnitude =
14      sqrtf(xAccel * xAccel + yAccel * yAccel + zAccel *
15           zAccel);
16
17  accelerationBaseline =
18      //Updates the baseline
19      acceleration to stay consistant with 1g of force
20      0.95f * accelerationBaseline +
21      0.05f * accelerationMagnitude;
22
23  accelerationFiltered =
24      fabsf(accelerationMagnitude - accelerationBaseline);
25      //Calculate difference
26      between acceleration magnitude and changing
27      baseline
28
29  bool cooldownComplete =
30      //Check if enough time has
31      passed to prevent multiple movements from counting
32      now - lastStepTime >= STEP_COOLDOWN_MS;
33
34  if (accelerationFiltered > STEP_THRESHOLD &&
35      cooldownComplete) {          //Check if step is
```

```
valid
27  if (lastStepTime > 0) {
                                //Check if there was a valid
                                step before calculating cadence
28  unsigned long stepInterval = now - lastStepTime;
                                //Calculate time between
                                steps
29
30  if (stepInterval > 0) {
                                //Prevents division by 0
31  float candidateCadence = 60000.0f / stepInterval;
                                //Convert time between
                                steps to steps per minute
32
33  if (candidateCadence >= MIN_CADENCE &&
                                //Accept cadence if it is
                                in between the valid range
34      candidateCadence <= MAX_CADENCE) {
35
36      latestMeasuredCadence = candidateCadence;
                                //Save cadence
37
38  if (workoutStarted) {
                                //Store cadence
                                measurments only if workout is active
39      addCadenceSample(candidateCadence);
40  }
```

```
Serial.print("[STEP] Measured cadence = ");  
//Serial monitor print  
  
Serial.print(candidateCadence, 1);  
  
Serial.print(" SPM, motion =");  
  
Serial.println(accelerationFiltered, 3);  
}  
}  
}  
  
lastStepTime = now;  
  
//Save step time for next  
cadence calculation  
}  

```

Heart-Rate-to-Cadence Controller

```
1 void updateCadenceController(unsigned long now) {
2     if (now - lastControlUpdate < CONTROL_INTERVAL_MS) {
3         //Check if enough time has
4         passed to update cadence
5         return;
6     }
7
8     lastControlUpdate = now;
9
10    //Save current time of update
```

```
7
8  Serial.println();
9  Serial.println("=====CONTROL_UPDATE=====");
10
11  if (heartRateCount < 2 || cadenceCount < 2) {
12      //Checks whether fewer than 2
13      valid hear rate and beat samples were collected
14      if (heartRateCount < 2) {
15          Serial.println("Not enough valid heart-rate samples.")
16          ; //Change heartRateCount/
17          cadenceCount instances to 1 if message keeps
18          appearing
19      }
20
21      if (cadenceCount < 2) {
22          //Check whether enough
23          cadence measurments were taking
24          Serial.println("Not enough valid cadence samples.");
25          //Appears when there is no
26          movement.
27      }
28
29      Serial.println("Keeping previous cadence command.");
30      //User is either matching
31      required cadence or no movement has happened to
32      change the cadence
```

```
21     resetMeasurementBuffers();
22     Serial.println("=====");
23     return;
24 }
25
26 float measuredHeartRate =
27     //Calculates the median of
28     valid heart ratesamples
29     calculateMedian(heartRateBuffer, heartRateCount);
30
31 float currentMeasuredCadence =
32     //Calculates the median of
33     valid cadence samples
34     calculateMedian(cadenceBuffer, cadenceCount);
35
36 float cadenceUsedByController =
37     //Decides whether to use valid
38     cadence otherwise use last cadence
39     previousCadenceValid
40     ? previousMeasuredCadence
41     : currentMeasuredCadence;
42
43 Serial.print("Target_HR:");
44     //Serial Monitor
45 Serial.println(targetHR, 1);
46 Serial.print("Measured_HR_median:");
47 Serial.println(measuredHeartRate, 1);
```

```
41 Serial.print("Measured_cadence_median:");
42 Serial.println(currentMeasuredCadence, 1);
43
44 float denominator = measuredHeartRate - restingHR;
                                //Calculate measured heartrate
                                difference
45 float numerator = targetHR - restingHR;
                                //Calculate desired heartrate
                                difference
46
47 if (denominator <= 5.0f || numerator <= 0.0f) {
                                //Rejects unvalid values should
                                the heartrate ratio equate to less than 40
48 Serial.println("Heart-rate_values_unsafe_for_controller.
    ");
                                //Debugging statement
49 Serial.println("Keeping_previous_cadence_command.");
                                //Similar case to no movement
                                resulting in no change in cadence
50 } else {
51 float heartRateRatio = numerator / denominator;
                                //Calculate multiplicative
                                heart rate ratio
52
53 float proposedCadence =
                                //Calculate new cadence
54 cadenceUsedByController * heartRateRatio;
```

```

56 proposedCadence =
//Ensure new cadence is still
with in valid cadence range
57 clampFloat(proposedCadence, MIN_CADENCE, MAX_CADENCE
);
58
59 proposedCadence =
//Makes sure range of change
in cadence doesn't explode
60 clampFloat(
61     proposedCadence,
62     commandedCadence - MAX_CADENCE_CHANGE,
63     commandedCadence + MAX_CADENCE_CHANGE
64 );
65
66 Serial.print("Cadence used by controller: ");
67 Serial.println(cadenceUsedByController, 1);
68 Serial.print("HR adjustment ratio: ");
69 Serial.println(heartRateRatio, 3);
70 Serial.print("Old command: ");
71 Serial.println(commandedCadence, 1);
72
73 commandedCadence = proposedCadence;
//Updates cadence
74 setMetronomeCadence(commandedCadence);
//Updates cadence time for

```

```
    audio

75
76    Serial.print("New command: ");
77    Serial.print(commandedCadence, 1);
78    Serial.println(" SPM");
79 }
80
81    previousMeasuredCadence = currentMeasuredCadence;
                                   //Saves cadence for next update
82    previousCadenceValid = true;
                                   //Sets Cadence as valid
83
84    resetMeasurementBuffers();
                                   //Clear buffers
85
86    Serial.println("=====");
87    Serial.println();
88 }
```

Appendix F

CAD Drawings

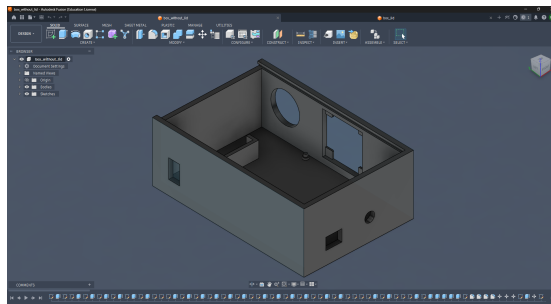


Figure 10: Main Enclosure

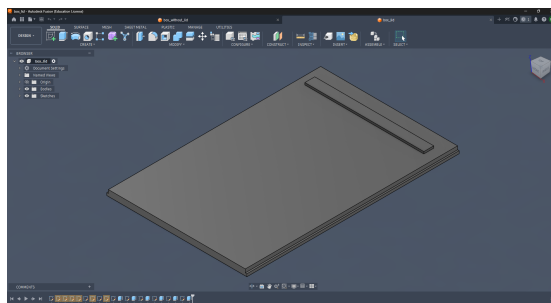


Figure 11: Main Enclosure Lid

Appendix G

Raw Test Data

G.1 PPG Sensor Data

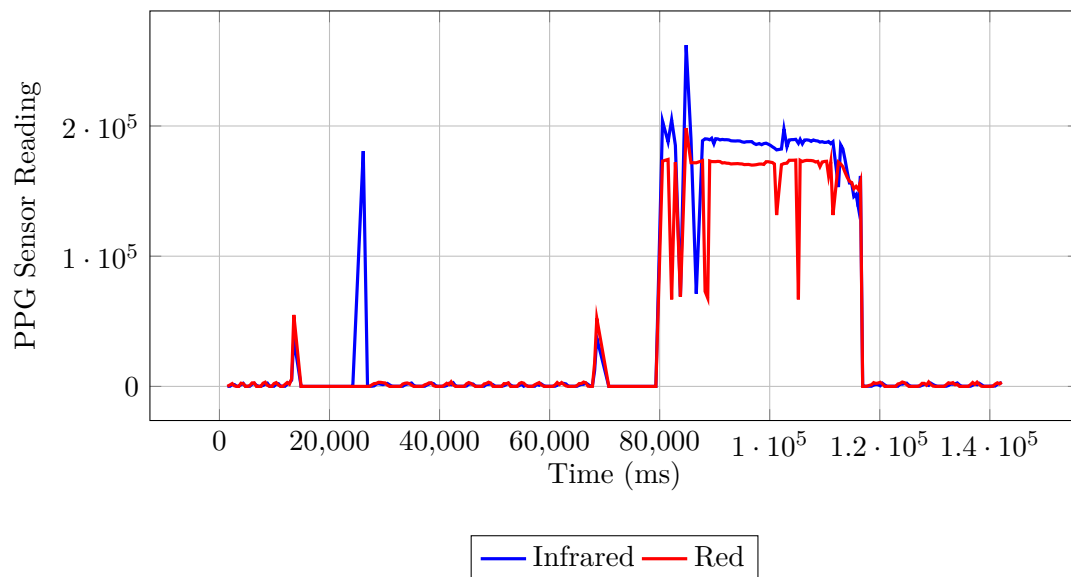


Figure 12: Raw infrared and red-light measurements collected from the MAX30102 sensor.

G.2 Accelerometer Data

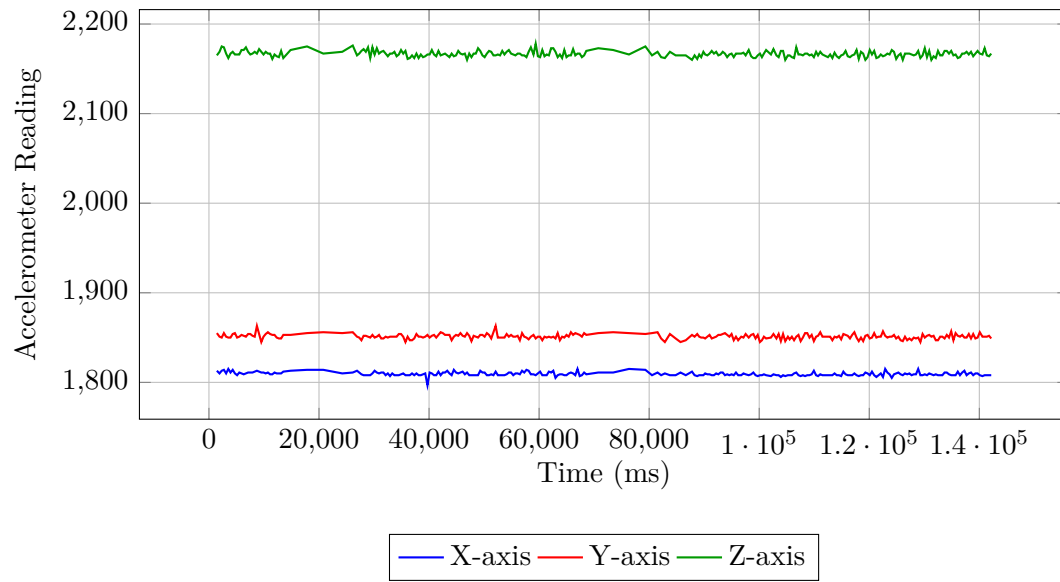


Figure 13: Raw three-axis measurements collected from the ADXL335 accelerometer.

Appendix H

Datasheets Referenced

Datasheets and reference documentation consulted during the design of this project are listed below by component.

- **ESP32 (microcontroller).** Espressif Systems, *ESP32 Series Datasheet*, Version 5.2, 2025 – see full citation [16] and the bibliography.
- **MAX30102 (PPG sensor).** Analog Devices/Maxim Integrated, *MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health*.
<https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf>
- **ADXL335 (accelerometer).** Analog Devices, *ADXL335: Small, Low Power, 3-Axis ± 3 g Accelerometer*, Rev. B. <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl335.pdf>
- **PCM5102A (audio DAC).** Texas Instruments, *PCM510xA 2.1 VRMS, 112/106/100 dB Audio Stereo DAC with PLL and 32-bit, 384 kHz PCM Interface*. Product page: <https://www.ti.com/product/PCM5102A>

- **SSD1306 (OLED display driver).** Solomon Systech, *SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Datasheet as hosted by Adafruit (vendor of the breakout board used): <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- **Rotary encoder (KY-040-style module).** This is a generic incremental rotary encoder module sold under many vendor names without a single canonical manufacturer datasheet; behavior was verified empirically against the pin functions described in Section 4.2 rather than a specific datasheet.

Appendix I

User Manual / Quick-Start Guide

Getting Started

1. **Attach the device.** Clip the enclosure to your waistband at the hip using the built-in belt clip, and place the PPG sensor on a finger.
2. **Power on.** Use the enclosure's power switch to turn the device on.
3. **Select your target heart rate.** On power-up, the OLED display shows "Select Heart Rate." Turn the rotary encoder knob to adjust the target value (60–200 BPM), shown in large digits in the center of the screen.
4. **Start your workout.** Once your target heart rate is set, press the rotary encoder knob to begin. The display switches to a live workout screen showing your target heart rate, current heart rate, measured cadence, and the commanded (metronome) cadence.
5. **Follow the beat.** Match your running cadence (footfalls) to the metronome tone played through your earbuds. The tempo will automatically speed up

or slow down roughly every 10 seconds to guide your heart rate toward your target.

6. **Stop your workout.** Press the rotary encoder knob again at any time to stop. The device returns to the heart-rate selection screen, ready to start another session.

Charging

The battery charges over USB-C through the port exposed on the side of the enclosure – there is no need to open the case to charge the device. A standard USB-C cable and any 5V USB power source (wall adapter, computer, power bank) can be used.

Replacing the Battery

If the battery ever needs to be replaced, slide open the enclosure’s lid and lift the battery out; a new battery of the same type can be dropped back into place. The enclosure is designed to hold the battery snugly once the lid is closed, so no additional fastening is required.

Tips

- Make sure the PPG sensor is snug against your finger – a loose fit is the most common cause of an inconsistent heart-rate reading.
- If the OLED shows “–” instead of a heart-rate value, the device has not detected a finger on the PPG sensor.

- The USB port on the ESP32 itself (accessible through a separate small cutout) is intended for firmware updates during development, not for normal charging or day-to-day use.